

Digital Design and Computer Organisation Laboratory

3rd Semester, Academic Year 2025

Date:29-9-2025

Name:	SRN:	Section

Week Number: 6

Program Number:

TITLE:

Deliverables

- I. Verilog Code Screenshot
- II. Verilog VVP Output Screen Shot
- III. GTKWAVE Screenshot
- IV. Output Table to be completed and included

2-bit Up Counter JK:

```
module jkff (input wire clk, input wire reset, input wire j, input wire k, output reg q );
    always @(posedge clk or posedge reset)
    begin
        if (reset)          q <= 1'b0;                // Reset to 0
        else begin case ({j, k})
            2'b00: q <= q;          // No change
            2'b01: q <= 1'b0;       // Reset
            2'b10: q <= 1'b1;       // Set
            2'b11: q <= ~q;         // Toggle
        endcase
    end
end

endmodule

// 2-bit synchronous up counter using JK flip-flops
module up_counter_2bit ( input clk, input reset, output [1:0] q);
    wire j0, k0, j1, k1;
    assign j0 = 1'b1;
    assign k0 = 1'b1;
    assign j1 = q[0];
    assign k1 = q[0];
    // Instantiate two JK flip-flops
    jkff jkff0 (.clk(clk), .reset(reset), .j(j0), .k(k0), .q(q[0]));
    jkff jkff1 (.clk(clk), .reset(reset), .j(j1), .k(k1), .q(q[1]));
endmodule
```

```

module tb_up_counter_2bit;
    reg clk, reset;
    wire [1:0] q;

    // Instantiate counter
    up_counter_2bit uut (.clk(clk), .reset(reset), .q(q));

    // Clock generation
    initial clk = 0;
    always #5 clk = ~clk;
// Clock period = 10
    initial
    begin
        $dumpfile("up_counter_jk.vcd");
        $dumpvars(0, tb_up_counter_2bit);
    end
    initial
    begin
        $display("Time\tClk\tReset\tQ1 Q0");
        $monitor("%0t\t%b\t%b\t%b %b", $time, clk, reset, q[1], q[0]);
        reset = 1; #10;    // Apply reset
        reset = 0;
        #60;                // Run counter for a while
        reset = 1; #10;    // Reset again
        reset = 0;
        #40;
        $finish;
    end
endmodule

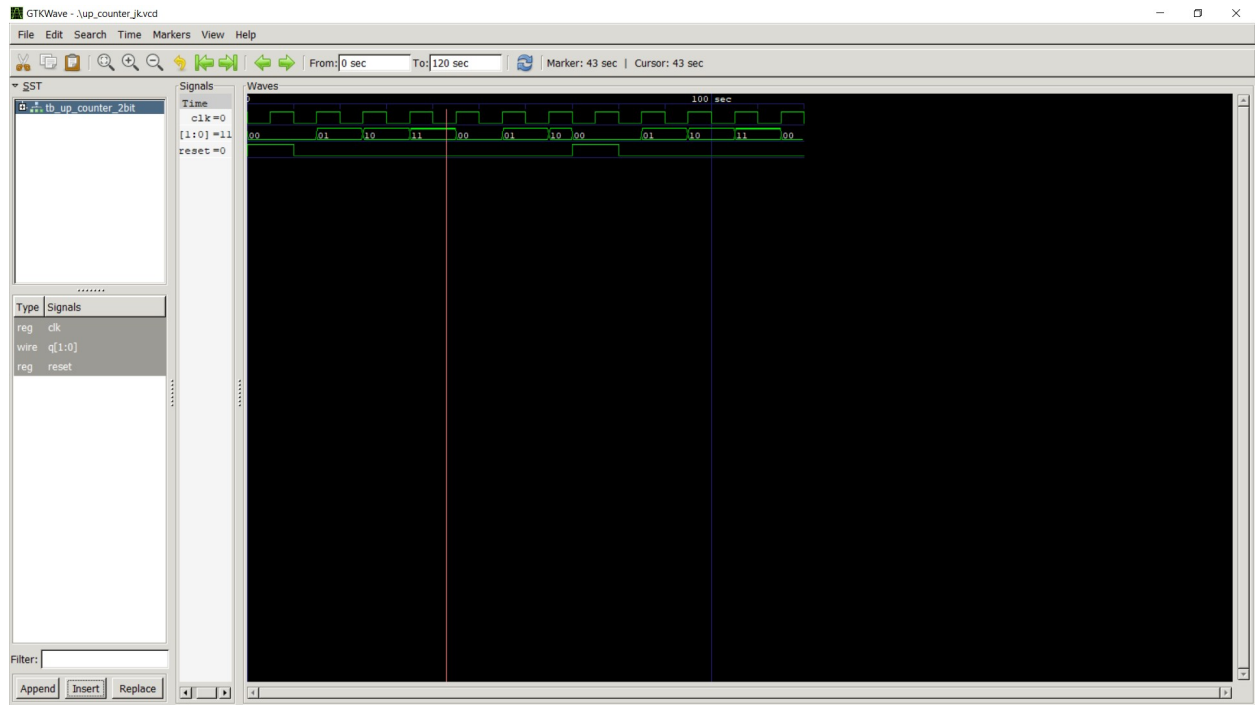
```

VCD info: dumpfile up_counter_jk.vcd opened for output.

Time	Clk	Reset	Q1	Q0
0	0	1	0	0
5	1	1	0	0
10	0	0	0	0
15	1	0	0	1
20	0	0	0	1
25	1	0	1	0
30	0	0	1	0
35	1	0	1	1
40	0	0	1	1
45	1	0	0	0
50	0	0	0	0
55	1	0	0	1
60	0	0	0	1
65	1	0	1	0
70	0	1	0	0
75	1	1	0	0
80	0	0	0	0
85	1	0	0	1
90	0	0	0	1
95	1	0	1	0
100	0	0	1	0
105	1	0	1	1
110	0	0	1	1
115	1	0	0	0

.\synupcounjktb.v:27: \$finish called at 120 (1s)

120	0	0	0	0
-----	---	---	---	---



2-bit Down Counter JK:

```
module jkff (input wire clk, input wire reset, input wire j, input wire k, output reg q );
    always @(posedge clk or posedge reset)
    begin
        if (reset)            q <= 1'b0;                // Reset to 0
        else begin case ({j, k})
            2'b00: q <= q;          // No change
            2'b01: q <= 1'b0;       // Reset
            2'b10: q <= 1'b1;       // Set
            2'b11: q <= ~q;        // Toggle
        endcase
    end
end

endmodule

// 2-bit synchronous down counter using JK flip-flops
module down_counter_2bit (input clk, input reset, output [1:0] q);
    // JK inputs
    wire j0, k0, j1, k1;
    assign j0 = 1'b1;
    assign k0 = 1'b1;
    assign j1 = ~q[0];
    assign k1 = ~q[0];

    // Instantiate flip-flops
    jkff jkff0 (.clk(clk), .reset(reset), .j(j0), .k(k0), .q(q[0]));
    jkff jkff1 (.clk(clk), .reset(reset), .j(j1), .k(k1), .q(q[1]));
endmodule
```

```

module tb_down_counter_2bit;
    reg clk, reset;
    wire [1:0] q;
    // Instantiate counter
    down_counter_2bit uut (.clk(clk), .reset(reset), .q(q));
    // Clock generation
    initial begin
        clk = 0;
        forever #5 clk = ~clk;    // Clock period = 10
    end
    initial begin
        $dumpfile("down_counter_jk.vcd");
        $dumpvars(0, tb_down_counter_2bit);
    end
    initial begin
        $display("Time\tClk\tReset\tQ1 Q0");
        $monitor("%0t\t%b\t%b\t%b %b", $time, clk, reset, q[1], q[0]);
        reset = 1; #10;    // Apply reset
        reset = 0;        #80;    // Run counter for a while
        reset = 1; #10;    // Reset again
        reset = 0;        #40;
        $finish;
    end
endmodule

```

VCD info: dumpfile down_counter_jk.vcd opened for output.

Time	Clk	Reset	Q1	Q0
------	-----	-------	----	----

0	0	1	0	0
---	---	---	---	---

5	1	1	0	0
---	---	---	---	---

10	0	0	0	0
----	---	---	---	---

15	1	0	1	1
----	---	---	---	---

20	0	0	1	1
----	---	---	---	---

25	1	0	1	0
----	---	---	---	---

30	0	0	1	0
----	---	---	---	---

35	1	0	0	1
----	---	---	---	---

40	0	0	0	1
----	---	---	---	---

45	1	0	0	0
----	---	---	---	---

50	0	0	0	0
----	---	---	---	---

55	1	0	1	1
----	---	---	---	---

60	0	0	1	1
----	---	---	---	---

65	1	0	1	0
----	---	---	---	---

70	0	0	1	0
----	---	---	---	---

75	1	0	0	1
----	---	---	---	---

80	0	0	0	1
----	---	---	---	---

85	1	0	0	0
----	---	---	---	---

90	0	1	0	0
----	---	---	---	---

95	1	1	0	0
----	---	---	---	---

100	0	0	0	0
-----	---	---	---	---

105	1	0	1	1
-----	---	---	---	---

110	0	0	1	1
-----	---	---	---	---

115	1	0	1	0
-----	---	---	---	---

120	0	0	1	0
-----	---	---	---	---

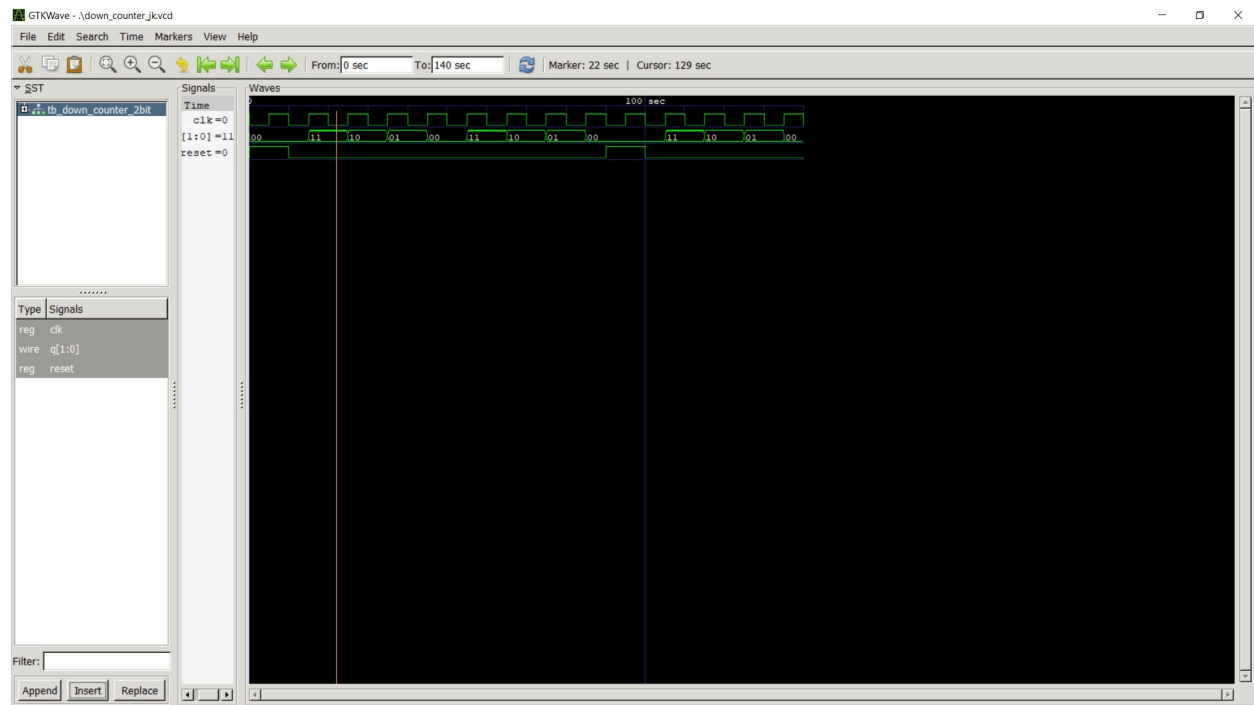
125	1	0	0	1
-----	---	---	---	---

130	0	0	0	1
-----	---	---	---	---

135	1	0	0	0
-----	---	---	---	---

.\syndocounjktb.v:22: \$finish called at 140 (1s)

140	0	0	0	0
-----	---	---	---	---



2-bit Up Down Counter JK:

```
module jkff (input wire clk, input wire rst, input wire j, input wire k, output reg q );
    always @(posedge clk or posedge rst)
    begin
        if (rst)
            q <= 1'b0; // Reset to 0
        else begin case ({j, k})
            2'b00: q <= q; // No change
            2'b01: q <= 1'b0; // Reset
            2'b10: q <= 1'b1; // Set
            2'b11: q <= ~q; // Toggle
        endcase
    end
end
endmodule

module updown_counter_2bit (input wire clk, input wire rst, input wire up_down, output wire [1:0] q);
    wire j0, k0, j1, k1;
    assign j0 = 1'b1;
    assign k0 = 1'b1;
    assign j1 = up_down * q[0] + ~up_down * q[0];
    assign k1 = up_down * q[0] + ~up_down * q[0];
    jkff jkff0 (.clk(clk), .rst(rst), .j(j0), .k(k0), .q(q[0]));
    jkff jkff1 (.clk(clk), .rst(rst), .j(j1), .k(k1), .q(q[1]));
endmodule
```

```

`timescale 1ns/1ps
module tb_updown_counter_2bit;
reg clk = 0;
reg rst = 1;
reg up_down = 1;
wire [1:0] q;
updown_counter_2bit dut (.clk(clk), .rst(rst), .up_down(up_down), .q(q));
// 10 ns period clock
always #5 clk = ~clk;
// Print after each synchronous update
always @(posedge clk) begin
| $display("%0t  clk=%0b rst=%0b upDn=%0b  Q=%0b %0b", $time, clk, rst, up_down, q[1], q[0]);
end
initial begin
| $monitor("%0t\t%b\t%b\t%b\t%b %b", $time, clk, rst, up_down, q[1], q[0]);
end

initial begin
| $dumpfile("updown_2bit_jk.vcd");
| $dumpvars(0, tb_updown_counter_2bit);
| // Keep reset high for one edge -> Q becomes 00 at first posedge
| @(negedge clk); rst = 1;
| @(negedge clk); rst = 0;          // release just before a posedge

| // Count UP to reach 11: 00 -> 01 -> 10 -> 11
| up_down = 1;
| repeat (3) @(negedge clk);      // change only on negedge

| // Now count DOWN: 11 -> 10 -> 01 -> 00
| up_down = 0;
| repeat (3) @(negedge clk);
| $finish;
end
endmodule

```

VCD info: dumpfile updown_2bit_jk.vcd opened for output.

0 0 1 1 0 0

5000 clk=1 rst=1 upDn=1 Q=0 0

5000 1 1 1 0 0

10000 0 1 1 0 0

15000 clk=1 rst=1 upDn=1 Q=0 0

15000 1 1 1 0 0

20000 0 0 1 0 0

25000 clk=1 rst=0 upDn=1 Q=0 0

25000 1 0 1 0 1

30000 0 0 1 0 1

35000 clk=1 rst=0 upDn=1 Q=0 1

35000 1 0 1 1 0

40000 0 0 1 1 0

45000 clk=1 rst=0 upDn=1 Q=1 0

45000 1 0 1 1 1

50000 0 0 0 1 1

55000 clk=1 rst=0 upDn=0 Q=1 1

55000 1 0 0 0 0

60000 0 0 0 0 0

65000 clk=1 rst=0 upDn=0 Q=0 0

65000 1 0 0 0 1

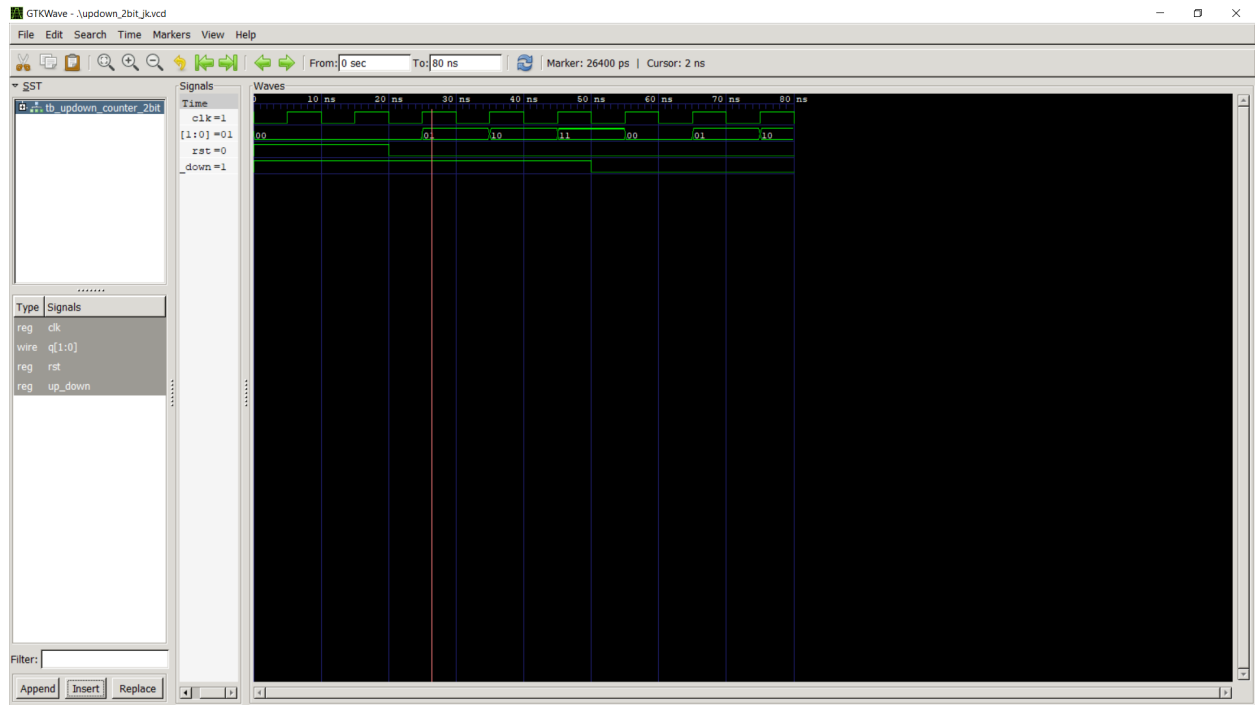
70000 0 0 0 0 1

75000 clk=1 rst=0 upDn=0 Q=0 1

75000 1 0 0 1 0

.\synupdocounjktb.v:32: \$finish called at 80000 (1ps)

80000 0 0 0 1 0



2-Bit Ripple Up Counter using DFF:

```
// Sequence: 00 → 01 → 10 → 11 → 00 → ...
`timescale 1ns/1ps
module dff (
    input wire clk,
    input wire rst,    // async reset, active-high
    input wire d,
    output reg q
);
    always @(posedge clk or posedge rst) begin
        if (rst) q <= 1'b0;
        else     q <= d;
    end
endmodule

module ripple_up_counter_2bit (
    input wire clk,
    input wire rst,
    output wire [1:0] q
);
    wire q0, q1;
    // LSB toggles every system clock
    dff ff0 (.clk(clk), .rst(rst), .d(~q0), .q(q0));

    // MSB toggles when q0 falls (use posedge of ~q0)
    dff ff1 (.clk(~q0), .rst(rst), .d(~q1), .q(q1));
    assign q = {q1, q0};
endmodule
```

```

module ripple_up_counter;
    reg clk, rst;
    wire [1:0] q;

    // Instantiate counter
    ripple_up_counter_2bit uut (.clk(clk), .rst(rst), .q(q));

    // Clock generation
    initial clk = 0;
    always #5 clk = ~clk;
    // Clock period = 10
    initial
    begin
        $dumpfile("ripple_up_counter_d.vcd");
        $dumpvars(0, ripple_up_counter);
    end
    initial
    begin
        $display("Time\tClk\tReset\tQ1 Q0");
        $monitor("%0t\t%b\t%b\t%b %b", $time, clk, rst, q[1], q[0]);
        rst = 1; #10;    // Apply reset
        rst = 0;
        #60;             // Run counter for a while
        rst = 1; #10;    // Reset again
        rst = 0;
        #40;
        $finish;
    end
endmodule

```

VCD info: dumpfile ripple_up_counter_d.vcd opened for output.

Time	Clk	Reset	Q1	Q0
------	-----	-------	----	----

0	0	1	0	0
---	---	---	---	---

5000	1	1	0	0
------	---	---	---	---

10000	0	0	0	0
-------	---	---	---	---

15000	1	0	0	1
-------	---	---	---	---

20000	0	0	0	1
-------	---	---	---	---

25000	1	0	1	0
-------	---	---	---	---

30000	0	0	1	0
-------	---	---	---	---

35000	1	0	1	1
-------	---	---	---	---

40000	0	0	1	1
-------	---	---	---	---

45000	1	0	0	0
-------	---	---	---	---

50000	0	0	0	0
-------	---	---	---	---

55000	1	0	0	1
-------	---	---	---	---

60000	0	0	0	1
-------	---	---	---	---

65000	1	0	1	0
-------	---	---	---	---

70000	0	1	0	0
-------	---	---	---	---

75000	1	1	0	0
-------	---	---	---	---

80000	0	0	0	0
-------	---	---	---	---

85000	1	0	0	1
-------	---	---	---	---

90000	0	0	0	1
-------	---	---	---	---

95000	1	0	1	0
-------	---	---	---	---

100000	0	0	1	0
--------	---	---	---	---

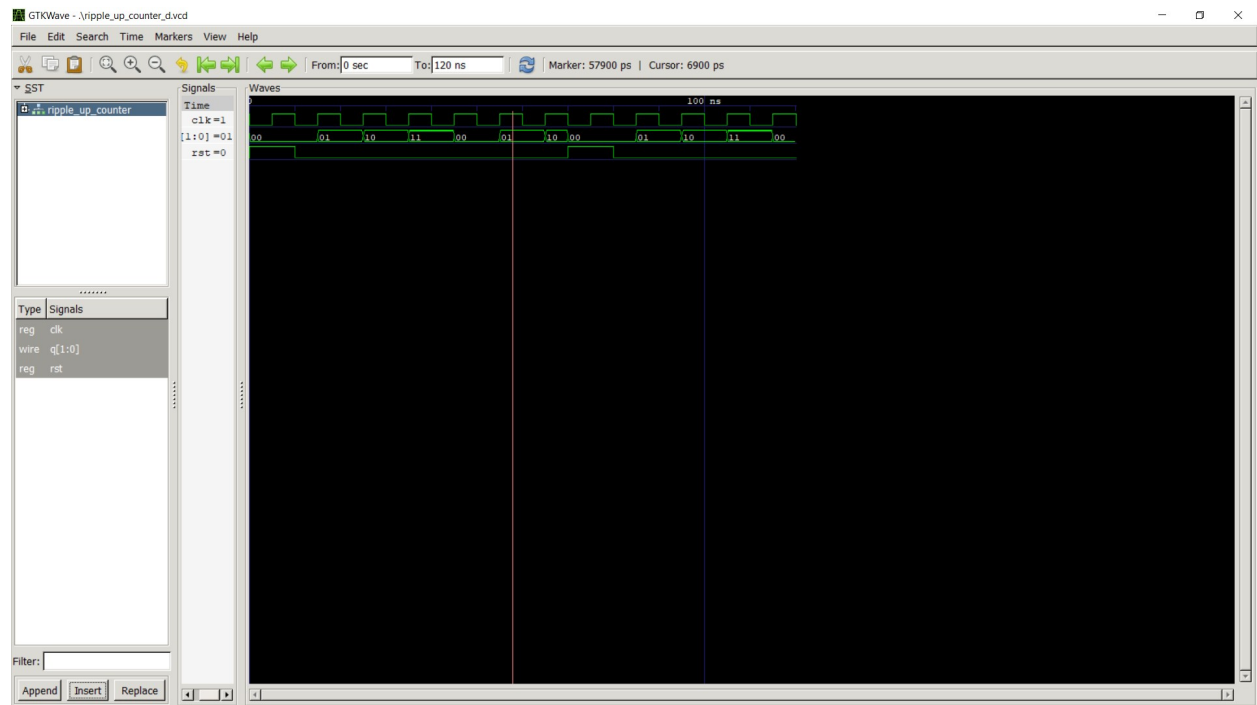
105000	1	0	1	1
--------	---	---	---	---

110000	0	0	1	1
--------	---	---	---	---

115000	1	0	0	0
--------	---	---	---	---

.\ripupdtb.v:27: \$finish called at 120000 (1ps)

120000	0	0	0	0
--------	---	---	---	---



3 bit Ring Counter:

```
`timescale 1ns/1ps
module ring_counter (
    input wire clk,
    input wire rst,          // async reset, active high
    output reg [2:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 3'b001;      // seed the ring with "001"
        else
            q <= {q[1:0], q[2]}; // rotate left
        end
    endmodule
```

```

`timescale 1ns/1ps
module tb_ring_counter;
    reg clk, rst;
    wire [2:0] q;

    // Instantiate DUT
    ring_counter uut (.clk(clk), .rst(rst), .q(q));

    // Clock generation: 10ns period
    always #5 clk = ~clk;

    initial begin
        |   $display("Time\tCLK\tRST\tQ2 Q1 Q0");
        end

    // Print on every rising edge
    always @(posedge clk) begin
        |   $display("%0t\t%b\t%b\t%b %b %b", $time, clk, rst, q[2], q[1], q[0]);
        end

    initial begin
        |   $dumpfile("ring_counter.vcd");
        |   $dumpvars(0, tb_ring_counter);

        |   clk = 0;
        |   rst = 1;    // Apply reset
        |   #10;
        |   rst = 0;    // Release reset

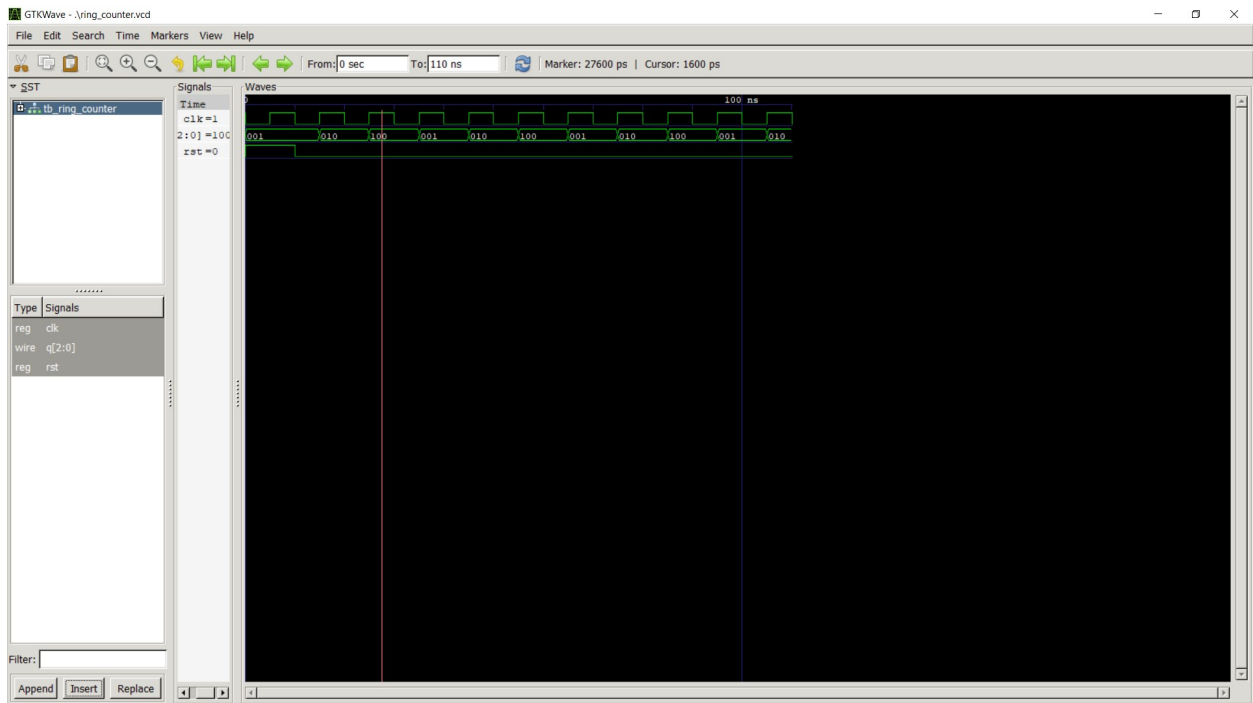
        |   □ #100;
        |   $finish;
        end
endmodule

```

```

VCD info: dumpfile ring_counter.vcd opened for output.
5000      1      1      0 0 1
15000     1      0      0 0 1
25000     1      0      0 1 0
35000     1      0      1 0 0
45000     1      0      0 0 1
55000     1      0      0 1 0
65000     1      0      1 0 0
75000     1      0      0 0 1
85000     1      0      0 1 0
95000     1      0      1 0 0
105000    1      0      0 0 1
.\ringdtb.v:31: $finish called at 110000 (1ps)

```



3 bit Johnson Counter:

```
`timescale 1ns/1ps
module johnson_counter (
    input wire clk,
    input wire rst,      // async reset, active high
    output reg [2:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 3'b000;          // Johnson starts at all zeros
        else
            q <= {q[1:0], ~q[2]};  // shift left, insert inverted MSB
    end
endmodule
```

```

`timescale 1ns/1ps
module tb_johnson_counter;
    reg clk, rst;
    wire [2:0] q;

    // Instantiate DUT
    johnson_counter uut (.clk(clk), .rst(rst), .q(q));

    // Clock generation: 10ns period
    always #5 clk = ~clk;

    initial begin
        $display("Time\tCLK\tRST\tQ2 Q1 Q0");
    end

    // Print on every rising edge
    always @(posedge clk) begin
        $display("%0t\t%b\t%b\t%b %b %b", $time, clk, rst, q[2], q[1], q[0]);
    end

    initial begin
        $dumpfile("johnson_counter.vcd");
        $dumpvars(0, tb_johnson_counter);

        clk = 0;
        rst = 1;    // Apply reset
        #10;
        rst = 0;    // Release reset

        #120;
        $finish;
    end
endmodule

```

```

VCD info: dumpfile johnson_counter.vcd opened for output
5000      1      1      0 0 0
15000     1      0      0 0 0
25000     1      0      0 0 1
35000     1      0      0 1 1
45000     1      0      1 1 1
55000     1      0      1 1 0
65000     1      0      1 0 0
75000     1      0      0 0 0
85000     1      0      0 0 1
95000     1      0      0 1 1
105000    1      0      1 1 1
115000    1      0      1 1 0
125000    1      0      1 0 0
.\johncountertb.v:31: $finish called at 130000 (1ps)

```

